

Hybrid RAG Knowledge Assistant

Major Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY:

ABHAY MAURYA (221620101001) AMIT YADAV (221620101010)

VARUN RANA (221620101059) SHANU AHMED (731620101003)

SAMIR AHMAD (731620101005)

Under the able guidance of

Assistant Prof. **Mrs. Sneh Lata Singh**



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DR. A.P.J. ABDUL KALAM GOVERNMENT INSTITUTE OF TECHNOLOGY
TANAKPUR (CHAMPAWAT, UTTARAKHAND) –262309**

(CAMPUS INSTITUTE OF)

**VEER MADHO SINGH BHANDARI UTTARAKHAND TECHNICAL
UNIVERSITY, DEHRADUN**

February – July, 2026



CERTIFICATE

This is to certify that the Major Project entitled “**Hybrid RAG Knowledge Assistant**” has been successfully completed and submitted by the following students:

Group Members	Roll No.
Abhay Maurya	221620101001
Amit Yadav	221620101010
Varun Rana	221620101059
Shanu Ahmed	731620101003
Samir Ahmad	731620101005

This Major Project is a record of the students’ original work carried out by them in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from **Dr. A.P.J. Abdul Kalam Government Institute of Technology, Tanakpur (Campus Institute of VMSBUTU, Dehradun), Uttarakhand, India.**

Mrs. Sneh Lata Singh

(Project Mentor)

Mr. Naresh Kumar

(Head of Department, CSE)

(External Examiner)

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our institute's director, **Prof. Hardwari Lal Mandoria**, whose knowledge and astute criticism have been crucial in forming this project and guaranteeing its successful conclusion.

We would like to express our deepest gratitude to our project mentor, **Mrs. Sneh Lata Singh**, for his valuable guidance, constant encouragement, and insightful suggestions throughout the development of our Major project "**Hybrid RAG Knowledge Assistant.**" His expertise, patience, and constructive feedback played a crucial role in shaping this project and bringing it to successful completion.

We extend our sincere thanks to **Mr. Naresh Kumar**, Head of the Department (CSE), for his continuous support, motivation, and for providing us with the facilities and environment necessary for the successful execution of this project.

Finally, we gratefully acknowledge the contributions of **Mr. Rishabh Kushawah, Mr. Divisth Jaiswal, Ms. Neha Bisht, Mrs. Komal Gahtori, Mr. Shubham Goyal, Mrs. Gagandeep Kaur and Mrs. Shreya Bisht** for their academic guidance and constant support during the entire course of this Major project.

We would also like to convey our heartfelt gratitude to our classmates, instructors, and friends for their cooperation, valuable inputs, and encouragement throughout this project. Their moral support and feedback have greatly contributed to the refinement of our work.

Name	Roll No.	Signature
Abhay Maurya	221620101001	
Amit Yadav	221620101010	
Varun Rana	221620101059	
Shanu Ahmed	731620101003	
Samir Ahmad	731620101005	

LIST OF FIGURES

4.2.1	System Architecture Diagram for Hybrid RAG Knowledge Assistant	20
5.8.1	Workflow Diagram for Hybrid RAG Knowledge Assistant	31
5.8.2	ER Diagram for Hybrid RAG Knowledge Assistant	31
5.9.1	Illustrates the homepage of the Hybrid RAG Knowledge Assistant.	32
5.9.2	Illustrates the Features of the Hybrid RAG Knowledge Assistant.	32
5.9.3	Illustrates the Dashboard of the Hybrid RAG Knowledge Assistant.	32
5.9.4	Illustrates the Document Library of the Hybrid RAG Knowledge Assistant.	33
5.9.5	Illustrates the Pricing page of the Hybrid RAG Knowledge Assistant.	33
5.9.6	Illustrates the Chat of the Hybrid RAG Knowledge Assistant.	33

ABBREVIATIONS

API	Application Programming Interface (Connects Frontend to Backend)
CSS	Cascading Style Sheets (Used for UI styling)
DB	Database (Stores users and chat logs)
ERD	Entity-Relationship Diagram (Visualizes data structure)
HTML	HyperText Markup Language (Used for web structure)
JS	JavaScript (Used for frontend logic and interactivity)
JSON	JavaScript Object Notation (Format for data exchange)
LLM	Large Language Model (The AI "brains" like Llama 3 or Gemini)
NLP	Natural Language Processing (The technology behind understanding text)
PDF	Portable Document Format (The primary document input type)
RAG	Retrieval-Augmented Generation (The core architecture of your project)
SDLC	Software Development Life Cycle (The Agile process used)
UI / UX	User Interface / User Experience (The web design and feel)
WBS	Work Breakdown Structure (The task hierarchy of your project)

ABSTRACT

The rapid advancement of Large Language Models (LLMs) has significantly improved natural language understanding and generation capabilities. However, these models often suffer from limitations such as hallucination, outdated knowledge, and the inability to access domain-specific information in real time. Retrieval-Augmented Generation (RAG) has emerged as an effective approach to address these challenges by integrating external knowledge retrieval with generative AI models. This project presents an Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework for Intelligent Question Answering, designed to improve the accuracy, relevance, and reliability of AI-generated responses.

The proposed framework combines sparse retrieval techniques, such as BM25, with dense retrieval methods based on vector embeddings to leverage the strengths of both keyword-based and semantic search. An adaptive retrieval mechanism dynamically selects the most suitable retrieval strategy based on query complexity and contextual requirements. Retrieved information from local databases, documents, and online knowledge sources is processed through a context fusion module, which ranks, filters, and merges relevant documents to construct high-quality contextual input for the Large Language Model.

The system further incorporates a feedback and continuous learning mechanism that enables performance improvement over time through user interactions and retrieval optimization. The framework supports both offline deployment, suitable for privacy-sensitive and resource-constrained environments, and online deployment, which provides access to real-time information and scalable computational resources. This flexibility makes the proposed solution applicable across various domains, including education, healthcare, enterprise knowledge management, research assistance, and intelligent chatbot systems.

The proposed Hybrid RAG framework aims to reduce hallucination, enhance contextual understanding, improve retrieval precision, and generate more accurate and trustworthy responses. By integrating adaptive retrieval strategies, context-aware information fusion, and advanced language models, the system demonstrates a scalable and efficient approach for intelligent question answering. The results indicate significant improvements in response quality, semantic relevance, and overall system performance compared with traditional retrieval and generation methods, making it a promising solution for next-generation AI-powered information retrieval and conversational systems.

Table of Contents

ACKNOWLEDGEMENT	i
LIST OF FIGURES	ii
ABBREVIATIONS	iii
ABSTRACT	iv
CHAPTER 1 INTRODUCTION	1
1.1 Background.....	1
1.2 Motivation.....	2
1.3 Problem Statement.....	2
1.4 Objectives.....	3
1.5 Scope of the Work.....	4
1.6 Contributions of the Project.....	5
CHAPTER 2 LITERATURE REVIEW	6
2.1 Introduction.....	6
2.2 Adaptive-RAG.....	6
2.3 RAG-LLM Survey.....	7
2.4 Dynamic Retrieval for RAG.....	8
2.5 Healthcare RAG Survey.....	9
2.6 Research Gap Discussion.....	10
CHAPTER 3 TECHNOLOGY AND METHODS	11
3.1 Introduction.....	11
3.2 Natural Language Processing.....	12
3.3 Information Retrieval Systems.....	12
3.4 Sparse Retrieval Techniques.....	13
3.4.1 TF-IDF.....	14
3.4.2 BM25.....	14
3.5 Dense Retrieval Techniques.....	14
3.5.1 Embeddings.....	15
3.5.2 Vector Search.....	15
3.6 Vector Databases.....	16
3.7 Large Language Models.....	17
3.8 Hallucination and Context Awareness.....	17
3.9 Work Breakdown Structure (WBS).....	18
3.10 Gantt Chart.....	18
CHAPTER 4 PROPOSED HYBRID RAG FRAMEWORK	19
4.1 Introduction.....	19
4.2 System Architecture.....	19
4.3 Query Processing Module.....	21
4.4 Hybrid Retrieval Engine.....	21
4.4.1 Sparse Retrieval (BM25).....	22
4.4.2 Dense Retrieval (Vector Search).....	22
4.4.3 Adaptive Retrieval Strategy.....	23
4.5 Results Merging and Reranking.....	23
4.6 Context Construction and Fusion.....	24
4.7 LLM-Based Response Generation.....	24

4.8 Advantages of the Proposed System	24
CHAPTER 5 SYSTEM DESIGN AND IMPLEMENTATION	26
5.1 Introduction.....	26
5.2 Development Environment	26
5.3 Hardware Requirements.....	27
5.4 Software Requirements	28
5.5 Embedding Model Configuration	28
5.6 Vector Database Configuration.....	29
5.7 LLM Integration.....	30
5.8 System Workflow Implementation	30
5.9 User Interface and Screenshots	31
CHAPTER 6 DEPLOYMENT MODELS	34
6.1 Introduction.....	34
6.2 Comparative Analysis	34
6.3 Security and Privacy Considerations	35
CHAPTER 7 RESULTS AND DISCUSSION	36
7.2 Evaluation Metrics	36
7.2.1 Precision	37
7.2.2 Recall	37
7.2.3 F1-Score.....	37
7.2.4 Response Relevance	38
7.2.5 Latency	38
7.3 Experimental Results	38
7.4 Comparative Performance Analysis.....	41
7.5 Discussion of Findings.....	41
7.6 Limitations of the System	41
CHAPTER 8 CONCLUSION AND FUTURE SCOPE.....	43
8.1 Conclusion	43
8.2 Key Contributions.....	44
8.3 Future Scope	44
8.4 Final Remarks	45
REFERENCES.....	46
APPENDIX A	48
Research Highlights	48
File Structure.....	48
APPENDIX B	50
Source Code	50
ABSTRACT IN HINDI	53
APPENDIX – C	54
VITAE.....	54
VITAE.....	55
VITAE.....	56
VITAE.....	57
VITAE.....	58

CHAPTER 1

INTRODUCTION

The Hybrid RAG Knowledge Assistant is a privacy-first document question-answering system designed to overcome the data security risks and hallucination issues associated with standard cloud-based AI. By utilizing Retrieval-Augmented Generation (RAG), the system allows users to upload custom documents and generates highly accurate answers based strictly on that provided context rather than pre-trained guesswork. Its core innovation is a dual-engine architecture that provides complete control over data privacy: an offline "Private Mode" that runs local models (like Ollama) for confidential files, ensuring sensitive information never leaves the device, and an online "Smart Mode" that leverages powerful cloud AI for faster, advanced reasoning on non-sensitive queries. Ultimately, this project delivers a secure, context-aware AI assistant that seamlessly balances uncompromising data privacy with cutting-edge artificial intelligence.

1.1 Background

The rapid evolution of Artificial Intelligence (AI) and Natural Language Processing (NLP) has transformed the way humans interact with information systems. Large Language Models (LLMs), such as GPT, Llama, and Claude, have demonstrated remarkable capabilities in understanding natural language, generating coherent responses, summarizing content, and assisting users in various knowledge-intensive tasks. These models are trained on vast amounts of textual data and possess the ability to generate human-like responses across multiple domains.

Despite their impressive performance, traditional LLMs face significant challenges. One of the major limitations is their reliance on static training data, which prevents them from accessing recent information or domain-specific knowledge not included during training. Furthermore, LLMs are susceptible to hallucination, a phenomenon where the model generates plausible but factually incorrect information. These limitations reduce the reliability of AI-generated responses in critical applications such as healthcare, education, research, and enterprise knowledge management.

1.2 Motivation

The rapid advancement of Artificial Intelligence (AI) and Large Language Models (LLMs) has transformed the field of natural language processing by enabling machines to understand and generate human-like text. Models such as GPT and Llama have demonstrated remarkable capabilities in tasks including question answering, content generation, summarization, and conversational assistance. Despite these achievements, traditional LLMs still face several challenges that limit their effectiveness in real-world applications. One of the most significant issues is their dependence on static training data, which prevents them from accessing recent information or domain-specific knowledge that was not available during training. As a result, these models may generate outdated, incomplete, or factually incorrect responses.

Another major challenge is hallucination, where the model produces information that appears convincing but lacks factual accuracy. This issue becomes particularly critical in domains such as healthcare, education, research, and enterprise knowledge management, where reliable information is essential for decision-making. Users increasingly expect AI systems to provide accurate, context-aware, and trustworthy responses rather than generic or potentially misleading answers.

1.3 Problem Statement

Large Language Models (LLMs) have significantly improved the capabilities of intelligent question-answering systems and conversational AI applications. However, despite their advanced language understanding and generation abilities, these models still face several limitations that affect their reliability and effectiveness in real-world environments. One of the primary challenges is their dependence on static training data, which prevents them from accessing newly available information after the training process is completed. Consequently, LLMs may provide outdated responses when users seek recent or dynamic information.

Another critical issue is hallucination, where the model generates information that appears accurate and convincing but is actually incorrect or unsupported by factual evidence. Such misleading responses can reduce user trust and may lead to serious consequences in domains such as healthcare, education, legal services, and enterprise decision-making. Additionally, traditional language models often struggle to provide context-aware responses for complex queries that require external knowledge or domain-specific information.

Existing Retrieval-Augmented Generation (RAG) systems attempt to address these limitations by combining information retrieval with response generation. However, many current approaches rely solely on either sparse retrieval techniques, such as BM25, or dense retrieval methods based on vector embeddings. Sparse retrieval is effective for exact keyword matching but often fails to capture semantic relationships between queries and documents. On the other hand, dense retrieval provides better semantic understanding but may overlook important keyword-based information. As a result, relying on a single retrieval strategy can lead to incomplete or less relevant retrieval results.

1.4 Objectives

The primary objective of this project is to design and develop an **Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework** for intelligent and context-aware question answering. The proposed system aims to overcome the limitations of traditional Large Language Models by integrating external knowledge retrieval with advanced language generation techniques. By combining multiple retrieval strategies and contextual understanding mechanisms, the framework seeks to improve the accuracy, relevance, and reliability of generated responses.

The specific objectives of this project are as follows:

- To develop a hybrid retrieval framework that integrates **BM25-based sparse retrieval** and **dense vector retrieval** for effective information retrieval.
- To implement an **adaptive retrieval mechanism** that dynamically selects the most suitable retrieval strategy based on query complexity and contextual requirements.
- To improve retrieval accuracy and semantic relevance by combining keyword-based matching with embedding-based semantic search techniques.
- To design a **context fusion and reranking module** that filters, prioritizes, and combines retrieved documents to provide high-quality contextual information.
- To integrate **Large Language Models (LLMs)** for generating accurate, coherent, and context-aware responses using retrieved knowledge.

- To reduce hallucination and improve factual consistency by grounding response generation with reliable external information sources.
- To support both **offline and cloud-based deployment environments** for enhanced flexibility, scalability, and practical usability.
- To evaluate the performance of the proposed framework using retrieval accuracy, response relevance, latency, and overall response quality metrics.

Through these objectives, the project aims to create a scalable and efficient intelligent question-answering system capable of delivering trustworthy and contextually relevant information across various real-world applications.

1.5 Scope of the Work

The scope of this project focuses on the design, development, and evaluation of an **Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework** for intelligent question answering and information retrieval. The proposed system combines sparse retrieval, dense vector retrieval, adaptive retrieval strategies, and Large Language Models to improve the quality and reliability of generated responses. The framework is intended to bridge the gap between traditional information retrieval systems and modern AI-based conversational systems by providing context-aware and factually grounded answers.

The project covers the implementation of a hybrid retrieval mechanism that integrates **BM25-based keyword retrieval** with **embedding-based semantic retrieval**. It also includes the development of context fusion and reranking techniques to ensure that the most relevant information is provided to the language model for response generation. The system supports both structured and unstructured data sources, including research papers, documents, knowledge bases, and web-based information repositories.

Another important aspect of the project is the support for **offline and cloud-based deployment environments**. Offline deployment enables secure processing of sensitive information within local infrastructure, while cloud deployment provides scalability, real-time information access, and efficient resource utilization. This flexibility allows the framework to be adapted for different operational requirements and application domains.

1.6 Contributions of the Project

This project contributes to the field of intelligent information retrieval and question-answering systems by proposing an **Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework** that combines advanced retrieval techniques with Large Language Models. The primary contribution of the proposed framework is the integration of **BM25-based sparse retrieval** and **dense vector retrieval**, enabling the system to leverage both keyword-based matching and semantic understanding for improved retrieval accuracy and relevance.

Another significant contribution is the implementation of an **adaptive retrieval mechanism** that dynamically selects the most appropriate retrieval strategy based on query complexity and contextual requirements. This approach improves the efficiency of information retrieval while ensuring that users receive more relevant and context-aware responses. The framework also introduces a **context fusion and reranking module** that combines information from multiple retrieved documents, filters irrelevant content, and provides high-quality contextual input to the language model.

The project further enhances response generation by integrating Large Language Models with retrieval-grounded knowledge, thereby reducing hallucination and improving factual correctness. Additionally, the framework supports both **offline and cloud-based deployment architectures**, making it suitable for a wide range of real-world applications with different security, privacy, and scalability requirements.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The rapid development of Large Language Models (LLMs) has significantly transformed the field of Natural Language Processing (NLP) by enabling advanced capabilities in text generation, question answering, summarization, and conversational intelligence. Models such as GPT, LLaMA, Claude, and PaLM have demonstrated remarkable performance across a wide range of language-related tasks. However, despite their success, these models suffer from several limitations, including hallucination, outdated knowledge, lack of explainability, and inability to access real-time information. These challenges have motivated researchers to explore approaches that combine language generation with external knowledge retrieval.

Retrieval-Augmented Generation (RAG) has emerged as one of the most promising solutions to address these limitations. By integrating information retrieval systems with generative language models, RAG enables access to external knowledge sources during response generation. This approach improves factual accuracy, enhances contextual understanding, and reduces the likelihood of generating misleading information. Over the past few years, numerous studies have proposed different RAG architectures, retrieval strategies, reranking mechanisms, and context fusion techniques to improve system performance.

Recent research has focused on hybrid retrieval methods that combine sparse retrieval techniques such as BM25 with dense retrieval approaches based on vector embeddings. These systems aim to leverage the strengths of both keyword matching and semantic understanding. Furthermore, adaptive retrieval frameworks have been introduced to dynamically select retrieval strategies according to query complexity and contextual requirements. Researchers have also investigated dynamic retrieval mechanisms, retrieval-generation collaboration, and feedback-based learning to enhance response quality and retrieval efficiency.

2.2 Adaptive-RAG

Jeong et al. (2024) proposed Adaptive-RAG, a novel retrieval-augmented framework designed to improve the efficiency and accuracy of Large Language Models (LLMs) by dynamically

selecting retrieval strategies based on question complexity. Traditional RAG systems typically apply a fixed retrieval process for all user queries, regardless of whether the query is simple, complex, or requires multi-step reasoning. This often leads to unnecessary computational overhead for straightforward questions and insufficient reasoning capabilities for complex multi-hop queries. To address this limitation, the authors introduced an adaptive mechanism that intelligently determines the most appropriate retrieval strategy before answer generation. The proposed framework employs a lightweight classifier to predict the complexity level of an incoming query. Based on this prediction, the system selects among three possible approaches: no retrieval for simple questions, single-step retrieval for moderately complex queries, and multi-step retrieval for complex reasoning tasks. This adaptive selection process enables the framework to balance retrieval effectiveness and computational efficiency. The authors automatically generated training labels for the classifier by utilizing model predictions and inherent dataset characteristics, eliminating the need for extensive manual annotation. The framework was evaluated on several open-domain question-answering benchmarks containing both single-hop and multi-hop questions. Experimental results demonstrated that Adaptive-RAG achieved superior performance compared to conventional retrieval-augmented approaches. The system improved answer accuracy while simultaneously reducing retrieval costs and response latency. Furthermore, the study highlighted the importance of considering query complexity when designing retrieval systems for Large Language Models.

2.3 RAG-LLM Survey

Fan et al. (2024) presented a comprehensive survey titled **“A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models”**, which provides an extensive review of the integration of Retrieval-Augmented Generation (RAG) techniques with Large Language Models (LLMs). The study highlights the growing importance of RAG systems in addressing fundamental limitations of LLMs, including hallucination, outdated knowledge, lack of domain-specific information, and difficulties in maintaining factual accuracy. By incorporating external knowledge sources into the generation process, Retrieval-Augmented Large Language Models (RA-LLMs) can generate more reliable, context-aware, and up-to-date responses.

The survey categorizes existing RA-LLM research into three major components: retrieval architecture, training strategies, and application domains. The authors discuss various retrieval methods, including sparse retrieval, dense retrieval, hybrid retrieval, and knowledge-enhanced

retrieval techniques. They further examine different training approaches such as fine-tuning, prompt engineering, in-context learning, and retrieval-aware optimization. The survey also explores a wide range of applications, including question answering, recommendation systems, scientific research, healthcare, software engineering, and domain-specific knowledge management.

One of the key contributions of the survey is its systematic classification of RAG architectures and identification of major challenges facing current RA-LLM systems. The authors emphasize issues such as retrieval quality, context length limitations, retrieval-generation alignment, computational efficiency, scalability, and hallucination mitigation. Additionally, the paper outlines several future research directions, including adaptive retrieval mechanisms, multimodal retrieval systems, dynamic knowledge integration, and more efficient retrieval-generation collaboration

2.4 Dynamic Retrieval for RAG

The study **“Context-Guided Dynamic Retrieval for Improving Generation Quality in RAG Models”** focuses on enhancing Retrieval-Augmented Generation (RAG) systems by introducing a dynamic retrieval mechanism that adapts the retrieval process according to the contextual requirements of a query. Traditional RAG frameworks generally retrieve a fixed number of documents using predefined retrieval strategies, regardless of the complexity or contextual needs of the input query. This often results in the retrieval of irrelevant information, increased computational costs, and reduced response quality. The authors address these challenges by proposing a context-aware retrieval approach that dynamically determines retrieval requirements during the generation process.

The proposed framework utilizes contextual information from both the user query and intermediate generation stages to guide the retrieval process. Instead of relying on a static retrieval pipeline, the system continuously evaluates the relevance and sufficiency of retrieved information. Based on this evaluation, additional retrieval operations can be triggered when necessary, ensuring that the language model receives the most relevant and contextually appropriate information. This dynamic retrieval strategy improves the collaboration between the retriever and generator, enabling more accurate and coherent responses.

Experimental evaluations demonstrate that context-guided retrieval significantly improves generation quality compared to conventional RAG models. The framework achieves better contextual relevance, higher factual accuracy, and reduced hallucination rates by ensuring that

retrieved knowledge remains aligned with the evolving generation context. Furthermore, the dynamic retrieval process enables efficient utilization of computational resources by retrieving information only when additional context is required.

2.5 Healthcare RAG Survey

Abo El-Enen et al. (2025) presented a comprehensive survey titled “**A Survey on Retrieval-Augmented Generation (RAG) Models for Healthcare Applications**”, which examines the role of Retrieval-Augmented Generation in addressing the unique challenges of healthcare information systems. The study highlights the growing adoption of Large Language Models (LLMs) in healthcare for tasks such as clinical decision support, medical question answering, diagnostics, treatment planning, and patient communication. However, the authors emphasize that traditional LLMs face significant limitations in healthcare environments, including hallucination, outdated medical knowledge, lack of transparency, and difficulties in providing evidence-based recommendations. To overcome these challenges, the survey explores how RAG architectures integrate external medical knowledge sources with language models to improve the reliability and accuracy of generated responses.

The survey categorizes healthcare RAG systems into three major architectures: **Naive RAG**, **Advanced RAG**, and **Modular RAG**. These architectures differ in terms of retrieval complexity, context construction, knowledge integration, and response generation capabilities. The authors discuss how retrieval mechanisms access trusted medical databases, clinical guidelines, research articles, and electronic health records to provide up-to-date and evidence-based information. The study further reviews various healthcare applications, including clinical documentation, diagnostic support, medical education, patient engagement, medical imaging analysis, and clinical research assistance.

A significant contribution of the survey is its analysis of ethical, regulatory, and practical challenges associated with deploying RAG systems in healthcare. Issues such as patient privacy, data security, bias mitigation, explainability, and compliance with healthcare regulations are extensively discussed. The authors also identify future research directions, including multimodal RAG systems, real-time knowledge integration, and specialty-specific medical retrieval architectures.

2.6 Research Gap Discussion

The literature review demonstrates that Retrieval-Augmented Generation (RAG) has emerged as an effective approach for improving the factual accuracy, contextual relevance, and reliability of Large Language Models (LLMs). Existing studies have proposed various retrieval mechanisms, adaptive retrieval strategies, and retrieval-generation integration techniques to overcome the limitations of traditional language models. However, despite significant advancements, several challenges remain unresolved.

Adaptive-RAG introduced a query-complexity-based retrieval mechanism that dynamically selects different retrieval strategies according to the complexity of incoming queries. While this approach improves computational efficiency and answer accuracy, its primary focus is limited to retrieval strategy selection. The framework does not extensively explore hybrid retrieval architectures that combine sparse retrieval and dense retrieval techniques for enhanced information retrieval performance.

Similarly, Context-Guided Dynamic Retrieval proposed a dynamic retrieval process that continuously adjusts retrieval behaviour based on generation context. Although the framework improves retrieval relevance and response quality, it mainly focuses on retrieval-generation collaboration and dynamic knowledge scheduling. The study provides limited attention to hybrid retrieval integration, advanced reranking mechanisms, and adaptive context fusion strategies that can further improve retrieval effectiveness.

Furthermore, the healthcare RAG survey demonstrates the importance of retrieval-grounded generation in high-stakes domains requiring accuracy, transparency, and evidence-based responses. However, many domain-specific RAG systems continue to face challenges related to knowledge updating, retrieval precision, explainability, and efficient deployment across diverse operational environment.

CHAPTER 3

TECHNOLOGY AND METHODS

3.1 Introduction

This chapter presents the theoretical concepts and technologies that form the foundation of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) framework. The effectiveness of modern intelligent question-answering systems depends on the integration of multiple disciplines, including Natural Language Processing (NLP), Information Retrieval (IR), Machine Learning, and Large Language Models (LLMs). These technologies enable systems to understand user queries, retrieve relevant information from external knowledge sources, and generate accurate and context-aware responses. This chapter discusses the fundamental principles of sparse and dense retrieval techniques, vector databases, language models, and the challenges of hallucination and contextual understanding. Understanding these concepts is essential for developing reliable, scalable, and efficient Retrieval-Augmented Generation systems.

Table: Technology Stack of the Proposed Adaptive Hybrid RAG Framework

Component	Technology
Frontend	HTML5, CSS3, JavaScript, Bootstrap
Backend	Java 17, Spring Boot, Maven
AI Service	Python, FastAPI, LangChain
Document Processing	PyPDFLoader, PyPDF
Text Chunking	RecursiveCharacterTextSplitter
Embeddings	Sentence Transformers (all-MiniLM-L6-v2)
Vector Database	FAISS
Retrieval	BM25 + FAISS + Cross-Encoder Reranker
Offline AI	Ollama
Online AI	OpenAI, Groq, Anthropic
Authentication	JWT
Version Control	Git, GitHub
Development Tools	Visual Studio Code (VS Code), Eclipse

Table: Technology Stack Used in the Proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework.

3.2 Natural Language Processing

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that focuses on enabling computers to understand, interpret, analyze, and generate human language in a meaningful manner. It combines concepts from computer science, linguistics, machine learning, and deep learning to facilitate effective communication between humans and machines. NLP plays a crucial role in modern intelligent systems, including virtual assistants, chatbots, machine translation systems, sentiment analysis tools, and question-answering applications.

The primary objective of NLP is to bridge the gap between human language and computer understanding. To achieve this, NLP systems perform various tasks such as tokenization, part-of-speech tagging, named entity recognition, syntactic parsing, semantic analysis, and text generation. Tokenization involves breaking text into smaller units called tokens, while named entity recognition identifies important entities such as people, locations, and organizations. Semantic analysis enables systems to understand the contextual meaning of words and sentences rather than relying solely on keyword matching.

Over the years, NLP has evolved from rule-based approaches to statistical and deep learning-based techniques. Traditional rule-based systems depended on predefined linguistic rules, which limited their flexibility and scalability. Statistical methods improved performance by learning language patterns from large datasets. More recently, transformer-based architectures such as BERT, GPT, and LLaMA have revolutionized NLP by providing advanced language understanding and generation capabilities.

3.3 Information Retrieval Systems

Information Retrieval (IR) is the process of searching, identifying, and retrieving relevant information from large collections of documents, databases, or digital repositories in response to a user query. The primary objective of an Information Retrieval System is to provide users with the most relevant information while minimizing irrelevant results. IR systems are widely used in search engines, digital libraries, enterprise knowledge management systems, recommendation platforms, and intelligent question-answering applications.

A typical Information Retrieval System consists of several components, including document collection, indexing, query processing, retrieval, ranking, and result presentation. During the indexing phase, documents are processed and converted into a structured format that allows

efficient searching. When a user submits a query, the system analyzes and processes the query before comparing it with the indexed documents. Relevant documents are then retrieved and ranked according to their similarity or relevance to the query.

Traditional IR systems rely on keyword-based retrieval methods such as Term Frequency-Inverse Document Frequency (TF-IDF) and BM25. These techniques evaluate document relevance based on the occurrence and importance of query terms within documents. While effective for exact keyword matching, traditional methods often struggle to understand semantic relationships between words and concepts.

Modern Information Retrieval Systems incorporate machine learning and deep learning techniques to enhance retrieval performance. Dense retrieval methods use vector embeddings to represent documents and queries in a semantic space, enabling systems to retrieve contextually relevant information even when exact keywords are absent. These advancements have significantly improved retrieval accuracy and user satisfaction.

3.4 Sparse Retrieval Techniques

Sparse retrieval techniques are traditional information retrieval methods that represent documents and queries using sparse vector representations. In these methods, each term in the vocabulary corresponds to a specific dimension in the vector space, and most dimensions contain zero values, resulting in a sparse representation. Sparse retrieval primarily relies on exact keyword matching and statistical analysis of term occurrences within documents to determine relevance.

These techniques have been widely used in search engines, digital libraries, and document retrieval systems due to their simplicity, efficiency, and interpretability. When a user submits a query, the retrieval system identifies documents containing matching terms and calculates relevance scores based on predefined statistical measures. Documents with higher relevance scores are ranked and presented to the user.

One of the major advantages of sparse retrieval is its effectiveness in handling keyword-based searches. It performs particularly well when users know the exact terms associated with the information they are seeking. Additionally, sparse retrieval methods are computationally efficient and require relatively low storage resources compared to complex neural retrieval systems. As a result, they remain widely adopted in many real-world applications

3.4.1 TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) is one of the most widely used statistical techniques in information retrieval and text mining. It is used to measure the importance of a term within a document relative to a collection of documents known as a corpus. The primary objective of TF-IDF is to identify words that are significant to a particular document while reducing the influence of commonly occurring terms.

TF-IDF consists of two components: **Term Frequency (TF)** and **Inverse Document Frequency (IDF)**. Term Frequency measures how often a term appears in a document. Words that occur frequently within a document are generally considered more important. Inverse Document Frequency measures the rarity of a term across the entire document collection. Terms that appear in many documents receive lower weights because they provide less discriminatory information.

3.4.2 BM25

Best Matching 25 (BM25) is an advanced probabilistic ranking algorithm widely used in modern information retrieval systems. It was developed as an improvement over traditional TF-IDF by incorporating document length normalization and more sophisticated term weighting mechanisms. BM25 is considered one of the most effective sparse retrieval techniques and serves as the backbone of many search engines and Retrieval-Augmented Generation (RAG) systems.

The BM25 algorithm evaluates document relevance by considering three key factors: term frequency within the document, inverse document frequency across the corpus, and document length. Unlike TF-IDF, BM25 prevents excessively long documents from receiving unfairly high relevance scores simply because they contain more occurrences of query terms. This normalization improves ranking accuracy and retrieval effectiveness.

3.5 Dense Retrieval Techniques

Dense retrieval techniques are modern information retrieval methods that represent documents and queries as dense numerical vectors generated by deep learning models. Unlike sparse retrieval techniques, which rely on exact keyword matching, dense retrieval focuses on capturing the semantic meaning and contextual relationships between words, sentences, and

documents. This capability allows retrieval systems to identify relevant information even when the query and document do not share the same keywords.

Dense retrieval is based on neural network architectures that transform textual data into low-dimensional vector representations known as embeddings. These embeddings encode semantic information, enabling similar concepts to be positioned close to one another in the vector space. As a result, dense retrieval systems can understand synonyms, related concepts, and contextual meanings more effectively than traditional keyword-based methods.

The retrieval process begins by converting both documents and user queries into embeddings using pre-trained language models such as BERT, Sentence-BERT, OpenAI Embeddings, or E5. Once embeddings are generated, similarity measures such as cosine similarity, Euclidean distance, or dot product are used to identify documents that are semantically closest to the query. This approach significantly improves retrieval accuracy for complex and context-dependent queries.

3.5.1 Embeddings

Embeddings are numerical vector representations of textual data that capture the semantic meaning and contextual relationships between words, sentences, or documents. In Natural Language Processing (NLP), embeddings transform text into dense vectors that can be processed efficiently by machine learning and deep learning models. Unlike traditional keyword-based representations, embeddings preserve semantic information, allowing systems to understand the similarity between different words and concepts.

Modern embedding models such as Word2Vec, GloVe, BERT, Sentence-BERT, OpenAI Embeddings, and E5 generate high-dimensional vectors that represent the contextual meaning of text. These models are trained on large datasets and learn linguistic patterns, enabling them to place semantically similar words and sentences closer together in vector space. For example, the words “car” and “automobile” may have similar vector representations despite being different terms.

3.5.2 Vector Search

Vector search is a retrieval technique that identifies relevant documents by comparing numerical vector representations, known as embeddings, rather than relying on exact keyword matching. It is a fundamental component of dense retrieval systems and enables semantic search by measuring the similarity between query embeddings and document embeddings.

The vector search process begins by converting both user queries and stored documents into embeddings using a pre-trained language model. These embeddings are then stored in a vector database. When a query is submitted, the system generates an embedding for the query and calculates similarity scores between the query vector and document vectors using metrics such as cosine similarity, Euclidean distance, or dot product. Documents with the highest similarity scores are retrieved and ranked as the most relevant results.

One of the major advantages of vector search is its ability to capture semantic relationships between words and concepts. Unlike traditional keyword-based retrieval methods, vector search can retrieve relevant information even when the query and documents use different terminology. This capability significantly improves retrieval performance for complex and context-dependent queries.

3.6 Vector Databases

Vector databases are specialized database systems designed to store, manage, and retrieve high-dimensional vector embeddings efficiently. Unlike traditional relational databases that organize information in rows and columns, vector databases are optimized for similarity search operations on dense vector representations generated by machine learning and deep learning models. These databases have become an essential component of modern Artificial Intelligence applications, particularly in semantic search, recommendation systems, and Retrieval-Augmented Generation (RAG) frameworks.

The primary purpose of a vector database is to enable fast and accurate retrieval of information based on semantic similarity. Documents, images, audio files, and other forms of data are converted into embeddings using pre-trained models and stored as vectors within the database. When a user submits a query, the query is also transformed into an embedding, and the database performs a similarity search to identify vectors that are closest to the query vector. This process allows the retrieval of contextually relevant information even when exact keywords are not present.

To achieve high retrieval performance, vector databases use advanced indexing techniques such as Approximate Nearest Neighbor (ANN) search, Hierarchical Navigable Small World (HNSW) graphs, and Inverted File Index (IVF). These indexing methods significantly reduce search time while maintaining high retrieval accuracy. Popular vector databases include Pinecone, ChromaDB, Weaviate, FAISS, Milvus, and Qdrant, which are widely used in AI-powered applications.

3.7 Large Language Models

Large Language Models (LLMs) are advanced artificial intelligence systems designed to understand, process, and generate human language. These models are trained on massive collections of textual data obtained from books, articles, websites, research papers, and other digital sources. By learning linguistic patterns, grammar, context, and semantic relationships, LLMs can perform a wide range of Natural Language Processing (NLP) tasks, including question answering, text summarization, language translation, content generation, sentiment analysis, and conversational interactions.

The development of Large Language Models has been driven by advancements in deep learning and the introduction of the Transformer architecture. Proposed by Vaswani et al. in 2017, the Transformer model utilizes self-attention mechanisms that enable efficient processing of sequential data while capturing long-range contextual dependencies. Unlike traditional recurrent neural networks, Transformers process text in parallel, significantly improving training efficiency and performance. Modern LLMs such as GPT, LLaMA, Claude, Gemini, and PaLM are built upon transformer-based architectures and contain billions of parameters that allow them to learn complex language representations.

LLMs typically undergo two major stages: pre-training and fine-tuning. During pre-training, the model learns general language patterns from large-scale text corpora using self-supervised learning techniques. Fine-tuning further adapts the model to specific tasks or domains using specialized datasets. This training process enables LLMs to generate coherent, contextually relevant, and human-like responses.

3.8 Hallucination and Context Awareness

Hallucination is one of the most significant challenges associated with Large Language Models (LLMs). It refers to the generation of information that appears convincing and grammatically correct but is factually incorrect, misleading, or unsupported by reliable evidence. Since LLMs generate responses by predicting the most probable sequence of words based on learned patterns rather than verifying facts from trusted sources, they may produce inaccurate or fabricated information. Hallucinations can occur due to limitations in training data, insufficient contextual information, ambiguous queries, or outdated knowledge within the model.

The impact of hallucination is particularly critical in domains such as healthcare, education, finance, and legal services, where inaccurate information can lead to poor decision-making and

serious consequences. For example, a language model may generate incorrect medical advice, provide inaccurate legal interpretations, or fabricate references that do not exist. Therefore, reducing hallucination has become a major focus of research in modern Artificial Intelligence systems.

Context awareness refers to the ability of an AI system to understand and utilize relevant contextual information while processing queries and generating responses. A context-aware system considers factors such as user intent, conversation history, domain knowledge, and retrieved external information to produce accurate and meaningful outputs. By understanding the context surrounding a query, the system can generate responses that are more relevant and aligned with user expectations.

3.9 Work Breakdown Structure (WBS)

Week No.	Activities Performed
Week 1	Selection of project topic and initial literature review
Week 2	Detailed literature review and identification of research gaps
Week 3	Requirement analysis and system architecture design
Week 4	Dataset collection, cleaning, and preprocessing
Week 5	Embedding generation and vector representation creation
Week 6	Vector database configuration and dense retrieval implementation
Week 7	Adaptive retrieval strategy and reranking implementation
Week 8	End-to-end workflow integration and implementation
Week 9	User interface development, testing, and performance evaluation
Week 10	Results analysis, report preparation, and documentation

3.10 Gantt Chart

Project Activity	Week 1-2	Week 3-4	Week 5-6	Week 7-8	Week 9-10
Topic Selection	■				
Literature Review	■	■			
Requirement Analysis		■			
System Design		■	■		
Data Collection		■	■		
Retrieval Module Development			■	■	
Vector Database Integration			■	■	
LLM Integration				■	
Testing & Evaluation				■	■
Documentation		■	■	■	■
Final Submission					■

CHAPTER 4

PROPOSED HYBRID RAG FRAMEWORK

4.1 Introduction

This chapter presents the proposed **Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework** developed for intelligent and context-aware question answering. The framework is designed to address the limitations of traditional Large Language Models, such as hallucination, outdated knowledge, and limited access to domain-specific information. By integrating external knowledge retrieval with advanced language generation capabilities, the proposed system aims to improve the accuracy, relevance, and reliability of AI-generated responses.

The proposed architecture combines both **sparse retrieval** and **dense retrieval** techniques to leverage the strengths of keyword-based and semantic search methods. An adaptive retrieval mechanism dynamically selects and optimizes retrieval strategies according to the characteristics of incoming queries. Retrieved information is further processed through reranking, context fusion, and knowledge integration modules before being provided to the Large Language Model for response generation.

4.2 System Architecture

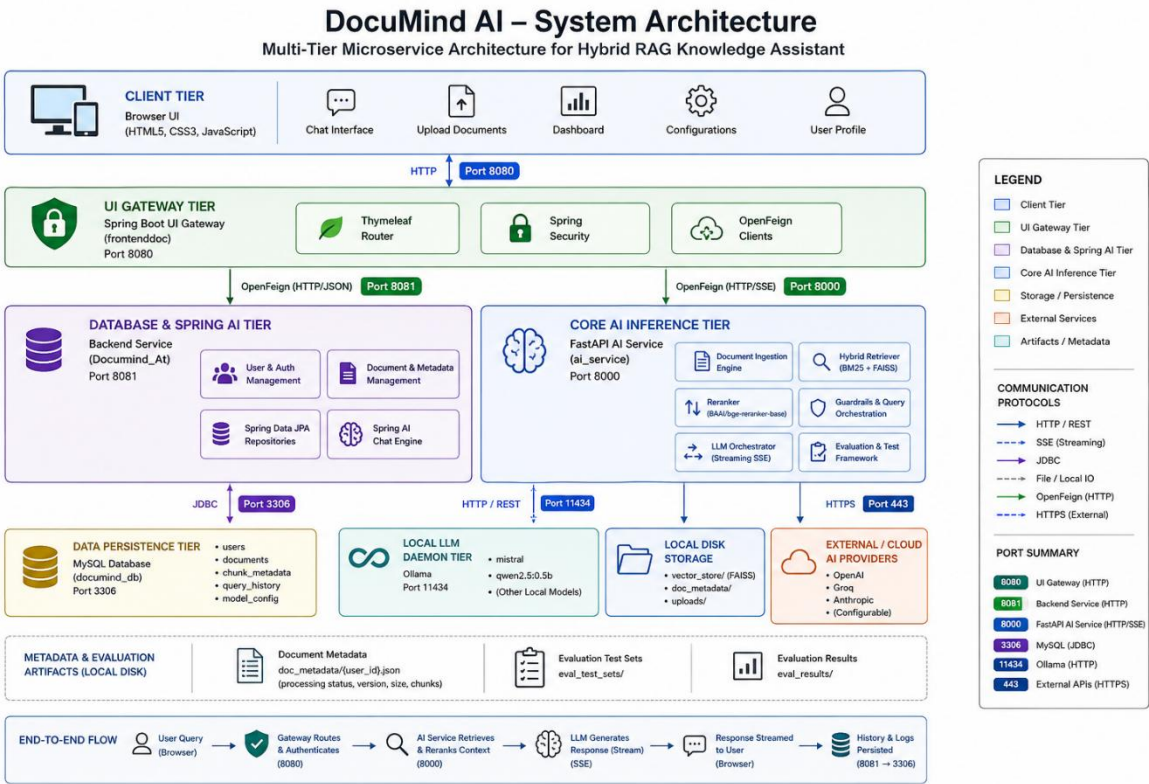
The proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework is designed to provide accurate, context-aware, and reliable responses by combining advanced information retrieval techniques with Large Language Models (LLMs). The architecture integrates sparse retrieval, dense retrieval, adaptive retrieval selection, context fusion, and language generation components within a unified framework. This design enables the system to efficiently retrieve relevant information from external knowledge sources and generate responses grounded in factual and up-to-date information.

The architecture begins with the **User Query Module**, where users submit questions or information requests through a user interface. The query is forwarded to the **Query Processing Module**, which performs preprocessing operations such as text normalization, tokenization,

stop-word removal, and query analysis. This stage helps improve retrieval effectiveness by transforming raw user input into a structured format suitable for retrieval operations.

After preprocessing, the query is passed to the **Hybrid Retrieval Engine**, which forms the core component of the proposed system. The retrieval engine consists of two parallel retrieval mechanisms: **Sparse Retrieval** and **Dense Retrieval**. The sparse retrieval component utilizes the BM25 algorithm to identify documents containing important query keywords. This approach is highly effective for exact term matching and document ranking based on lexical relevance. Simultaneously, the dense retrieval component converts the query into vector embeddings and performs semantic search using a vector database. This allows the system to retrieve documents that are contextually similar to the query, even when exact keywords are not present.

The outputs of both retrieval mechanisms are analyzed by the **Adaptive Retrieval Strategy Module**. This module evaluates query characteristics and dynamically determines the most appropriate retrieval combination. Depending on query complexity and contextual requirements, the system may prioritize sparse retrieval, dense retrieval, or a combination of both. This adaptive mechanism improves retrieval efficiency while maximizing relevance and coverage.



Figures 4.2.1: System Architecture

4.3 Query Processing Module

The Query Processing Module is the first operational component of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. Its primary purpose is to analyze, preprocess, and transform user queries into a structured format that can be efficiently processed by the retrieval and generation components of the system. Since user queries may contain grammatical variations, spelling inconsistencies, ambiguous terms, or unnecessary words, effective query processing is essential for improving retrieval accuracy and response quality.

The process begins when a user submits a query through the system interface. The query is first subjected to **text normalization**, which converts all text into a standardized format. This step may include converting characters to lowercase, removing special symbols, eliminating extra spaces, and correcting basic formatting inconsistencies. Text normalization ensures consistency and reduces variations that may negatively affect retrieval performance.

Following normalization, the system performs **tokenization**, which divides the query into smaller units known as tokens. These tokens may represent words, phrases, or meaningful linguistic elements. Tokenization enables the system to analyze individual components of the query and prepare them for further processing. After tokenization, **stop-word removal** is applied to eliminate common words such as “the,” “is,” “and,” and “of,” which generally contribute little semantic value to retrieval operations. Removing such words reduces processing complexity and improves retrieval efficiency.

The module then performs **query analysis and intent understanding**. During this stage, the system identifies important keywords, entities, and contextual information contained within the query.

4.4 Hybrid Retrieval Engine

The Hybrid Retrieval Engine is the core component of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. Its primary responsibility is to retrieve the most relevant information from external knowledge sources by combining the strengths of both sparse retrieval and dense retrieval techniques. Traditional retrieval systems often rely on a single retrieval method, which may limit their ability to handle diverse query types effectively. Sparse retrieval methods excel at exact keyword matching but often fail to capture semantic relationships, whereas dense retrieval methods provide strong semantic understanding but may

overlook important keyword-specific information. To overcome these limitations, the proposed framework integrates both approaches within a unified retrieval architecture.

When a processed query is received from the Query Processing Module, it is simultaneously forwarded to the sparse retrieval and dense retrieval components. The sparse retrieval module uses the BM25 ranking algorithm to identify documents containing important query terms and rank them according to lexical relevance.

4.4.1 Sparse Retrieval (BM25)

Sparse retrieval forms one of the fundamental components of the proposed Hybrid Retrieval Engine. In the proposed framework, the **Best Matching 25 (BM25)** algorithm is employed as the primary sparse retrieval technique due to its effectiveness in keyword-based document retrieval and ranking. BM25 is a probabilistic information retrieval model that evaluates the relevance of documents by considering query term frequency, inverse document frequency, and document length normalization. It is widely recognized as one of the most effective traditional retrieval algorithms and continues to serve as the backbone of many modern search systems.

When a user query is received and processed, the BM25 retrieval module analyzes the query terms and searches the indexed document collection for matching keywords. Each document is assigned a relevance score based on the occurrence and importance of query terms within the document. Terms that appear frequently in a document but are relatively rare across the entire document collection receive higher weights.

4.4.2 Dense Retrieval (Vector Search)

Dense retrieval is the second major component of the proposed Hybrid Retrieval Engine and is responsible for performing semantic information retrieval. Unlike sparse retrieval techniques that rely on exact keyword matching, dense retrieval uses vector embeddings to capture the semantic meaning and contextual relationships between queries and documents. This capability enables the system to retrieve relevant information even when the query and documents use different vocabulary but convey similar meanings.

In the proposed framework, user queries are converted into high-dimensional vector embeddings using a pre-trained embedding model. Similarly, all documents within the knowledge base are transformed into embeddings and stored in a vector database. These

embeddings represent the semantic characteristics of textual content and allow the retrieval system to compare documents based on meaning rather than lexical similarity.

When a query is submitted, the generated query embedding is compared with document embeddings stored in the vector database. Similarity metrics such as cosine similarity are used to measure the distance between vectors. Documents with the highest similarity scores are identified as the most semantically relevant and are retrieved for further processing. This semantic retrieval capability enables the system to identify contextually related information even when exact query keywords are absent.

4.4.3 Adaptive Retrieval Strategy

The Adaptive Retrieval Strategy is one of the most important innovations within the proposed Hybrid Retrieval-Augmented Generation framework. Its primary objective is to dynamically determine the most suitable retrieval approach for each incoming query by analyzing query characteristics, retrieval requirements, and contextual complexity. Unlike conventional retrieval systems that rely on a fixed retrieval mechanism, the adaptive strategy intelligently combines sparse retrieval and dense retrieval to maximize retrieval effectiveness and response quality.

After receiving retrieval results from both the BM25-based sparse retrieval module and the vector search-based dense retrieval module, the adaptive strategy evaluates several factors.

4.5 Results Merging and Reranking

The Results Merging and Reranking Module plays a crucial role in the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework by combining and refining the documents retrieved from both sparse and dense retrieval mechanisms. Since the Hybrid Retrieval Engine retrieves information from multiple sources using different retrieval strategies, the resulting document sets may contain duplicate entries, overlapping information, or documents with varying levels of relevance. Therefore, an effective merging and reranking process is essential to ensure that only the most useful and contextually relevant information is passed to subsequent stages of the system.

The merging process begins by collecting retrieval outputs from the BM25-based sparse retrieval module and the vector search-based dense retrieval module. Duplicate documents retrieved by both methods are identified and removed to prevent redundancy. The remaining

documents are consolidated into a unified result set that contains information from both lexical and semantic retrieval processes.

4.6 Context Construction and Fusion

The Context Construction and Fusion Module is a critical component of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. Its primary function is to transform the ranked retrieval results into a structured and coherent context that can be effectively utilized by the Large Language Model (LLM) during response generation. Since the retrieval process may return multiple documents containing overlapping, complementary, or partially relevant information, an efficient context construction mechanism is required to organize and integrate this information before it is supplied to the language model.

The process begins by receiving the reranked documents from the Results Merging and Reranking Module.

4.7 LLM-Based Response Generation

The LLM-Based Response Generation Module is the final processing component of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. Its primary responsibility is to generate accurate, coherent, and context-aware responses by utilizing the fused context produced by the retrieval and context construction modules. This module combines the reasoning and language generation capabilities of Large Language Models (LLMs) with externally retrieved knowledge to produce reliable and informative answers.

The process begins when the fused context, along with the original user query, is provided as input to the Large Language Model. Unlike conventional language models that rely solely on their internal training knowledge, the proposed system grounds response generation using retrieved information obtained from external knowledge sources. This retrieval-grounded approach enables the model to generate responses that are more accurate, up-to-date, and relevant to the user's query.

4.8 Advantages of the Proposed System

The proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework offers several advantages over traditional information retrieval systems and standalone Large Language Models. By combining sparse retrieval, dense retrieval, adaptive retrieval strategies,

and retrieval-grounded response generation, the system provides improved retrieval effectiveness and response quality. One of the major advantages of the framework is its ability to balance keyword-based retrieval and semantic search.

CHAPTER 5

SYSTEM DESIGN AND IMPLEMENTATION

5.1 Introduction

This chapter describes the design and implementation of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. It presents the technologies, tools, datasets, and system components used to develop the intelligent question-answering system. The chapter explains the development environment, hardware and software requirements, knowledge sources, embedding models, vector database configuration, and Large Language Model integration. It also discusses the implementation workflow, including query processing, hybrid retrieval, context fusion, and response generation. Furthermore, the chapter provides an overview of the user interface and system operation. These implementation details demonstrate how the proposed framework was developed to deliver accurate, context-aware, and reliable responses through the integration of retrieval mechanisms and advanced language models.

5.2 Development Environment

The development environment provides the hardware and software infrastructure required for designing, implementing, testing, and deploying the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. A suitable development environment is essential for ensuring efficient system performance, scalability, and seamless integration of retrieval mechanisms with Large Language Models.

The proposed system was developed primarily using **Python**, which was selected due to its extensive support for Artificial Intelligence, Machine Learning, Natural Language Processing, and Information Retrieval applications. Python provides a rich ecosystem of libraries and frameworks that simplify the implementation of advanced retrieval and generation functionalities.

The development process was carried out using **Visual Studio Code (VS Code)** and **Jupyter Notebook**. VS Code served as the primary Integrated Development Environment (IDE) for coding, debugging, and project management, while Jupyter Notebook was utilized for experimentation, model evaluation, and data analysis. The system was developed and tested on

the **Windows operating system**, although the framework is compatible with Linux-based environments as well.

5.3 Hardware Requirements

The successful implementation and execution of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework require adequate hardware resources to support data processing, embedding generation, vector retrieval, and Large Language Model integration. The hardware configuration should provide sufficient computational power, memory, and storage to ensure smooth system performance and efficient retrieval operations. A modern multi-core processor such as an **Intel Core i5/i7** or **AMD Ryzen 5/7** is recommended for handling query processing and retrieval tasks. A minimum of **8 GB RAM** is required for system operation, while **16 GB RAM** is recommended for improved performance when working with large document collections and embedding models. Storage requirements depend on the size of the knowledge base, with a minimum of **256 GB SSD** recommended for faster data access and retrieval.

Although the proposed framework can operate on CPU-based systems, a dedicated **Graphics Processing Unit (GPU)** can significantly accelerate embedding generation, vector computations, and language model inference. Stable internet connectivity is also required when utilizing cloud-based APIs and external Large Language Models.

Hardware Specifications

Component	Recommended Specification
Processor	Intel Core i5/i7 or AMD Ryzen 5/7
RAM	8 GB (Minimum), 16 GB (Recommended)
Storage	256 GB SSD or higher
GPU	NVIDIA GPU (Optional)
Internet Connectivity	Required for API Access
Operating System	Windows 10/11 or Linux

The above hardware configuration provides sufficient resources for efficient deployment and operation of the proposed Hybrid RAG system.

5.4 Software Requirements

The proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework requires a combination of software tools, libraries, and frameworks to support system development, retrieval operations, embedding generation, vector database management, and response generation. These software components provide the necessary environment for implementing and evaluating the proposed intelligent question-answering system.

Python 3.10 or higher serves as the primary programming language due to its extensive support for Artificial Intelligence, Machine Learning, and Natural Language Processing applications. **Visual Studio Code (VS Code)** and **Jupyter Notebook** are used for coding, testing, debugging, and experimentation. For retrieval and language model integration, frameworks such as **LangChain** and **LlamaIndex** facilitate efficient management of retrieval pipelines and contextual information flow.

Software Specifications

Software/Tool	Purpose
Python 3.10+	Core Programming Language
VS Code	Development Environment
Jupyter Notebook	Testing and Experimentation
LangChain	Retrieval Pipeline Management
Sentence Transformers	Embedding Generation
ChromaDB / FAISS	Vector Database
OpenAI API / Gemini API	LLM Integration
Streamlit / Gradio	User Interface Development

These software components collectively provide a reliable and scalable environment for implementing the proposed Hybrid RAG framework.

5.5 Embedding Model Configuration

The embedding model plays a fundamental role in the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework by converting textual information into dense numerical vector representations. These embeddings capture semantic relationships and contextual meanings within documents, enabling efficient semantic retrieval. The quality of embeddings directly influences retrieval accuracy and the relevance of generated responses.

In the proposed system, a pre-trained embedding model such as **Sentence-BERT (SBERT)** or **OpenAI Embeddings** is utilized to generate vector representations for both user queries and knowledge documents. These models are specifically designed to capture semantic similarity between textual inputs and produce high-quality embeddings suitable for information retrieval tasks.

The embedding generation process begins after document preprocessing and chunking. Large documents are divided into smaller text segments to ensure efficient processing and improved contextual representation. Each text chunk is then passed through the embedding model, which converts the text into a high-dimensional vector. These vectors preserve semantic information and allow documents with similar meanings to be positioned close together in vector space. Similarly, when a user submits a query, the query is transformed into an embedding using the same model. The generated query vector can then be compared with stored document vectors to identify semantically relevant information. Consistent use of the same embedding model for both documents and queries ensures accurate similarity calculations.

5.6 Vector Database Configuration

The vector database serves as the primary storage and retrieval infrastructure for dense retrieval operations within the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. It stores vector embeddings generated from knowledge documents and enables efficient semantic search based on similarity measurements. The use of a vector database allows the system to retrieve contextually relevant information quickly, even from large document collections.

In the proposed framework, **ChromaDB** is utilized as the vector database due to its simplicity, scalability, and compatibility with modern retrieval frameworks. ChromaDB provides efficient storage, indexing, and retrieval of high-dimensional embeddings generated by the embedding model. Alternative vector databases such as FAISS, Pinecone, Weaviate, or Milvus can also be integrated depending on deployment requirements.

After embedding generation, each document chunk is stored within the vector database along with its associated metadata. Metadata may include document title, source, category, creation date, and chunk identifiers. This additional information facilitates filtering and improves retrieval precision during search operations.

5.7 LLM Integration

Large Language Model (LLM) integration is a key component of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework. The LLM is responsible for interpreting user queries, processing retrieved contextual information, and generating coherent, accurate, and context-aware responses. By combining retrieval mechanisms with advanced language generation capabilities, the framework overcomes many limitations associated with standalone language models.

The proposed system integrates a pre-trained Large Language Model such as **GPT-4**, **Gemini**, or **LLaMA** through an API-based architecture. The integration process begins after the retrieval, reranking, and context fusion stages are completed. The selected contextual information is combined with the original user query to form a structured prompt that is provided to the language model.

Prompt engineering plays an important role in the integration process. The prompt is carefully designed to include the user query, retrieved context, and response instructions. This structured input helps the language model focus on relevant information and generate responses that align with the retrieved knowledge. By grounding the generation process in external information, the system significantly reduces hallucination and improves factual accuracy.

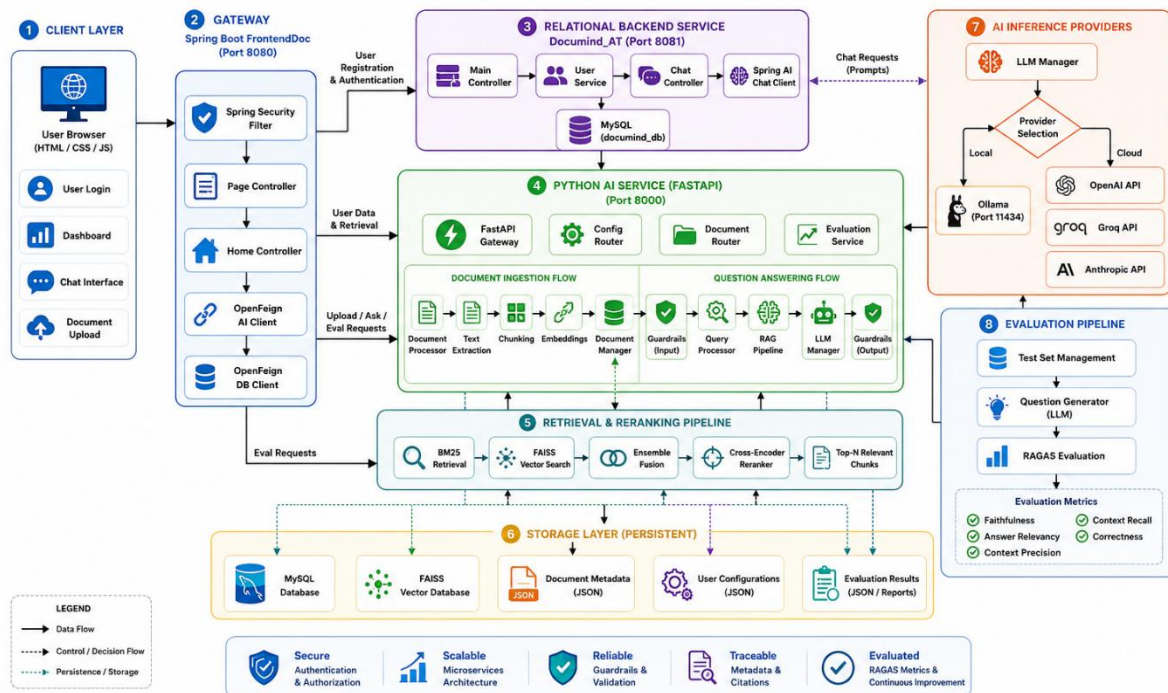
5.8 System Workflow Implementation

The implementation workflow of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework consists of a sequence of interconnected stages that enable intelligent question answering. The process begins when a user submits a query through the system interface. The Query Processing Module analyzes and preprocesses the query by performing text normalization, tokenization, and intent identification.

The processed query is then forwarded to the Hybrid Retrieval Engine, where both BM25-based sparse retrieval and vector-based dense retrieval are executed simultaneously. Relevant documents obtained from these retrieval mechanisms are merged and reranked according to their relevance scores. The Context Construction and Fusion Module then extracts important information and combines it into a structured context.

The fused context, along with the original user query, is provided to the Large Language Model (LLM). Using retrieval-grounded knowledge, the LLM generates a coherent, context-aware, and factually accurate response.

DocuMind AI – End-to-End Data Flow Architecture



Figures 5.8.1 Workflow

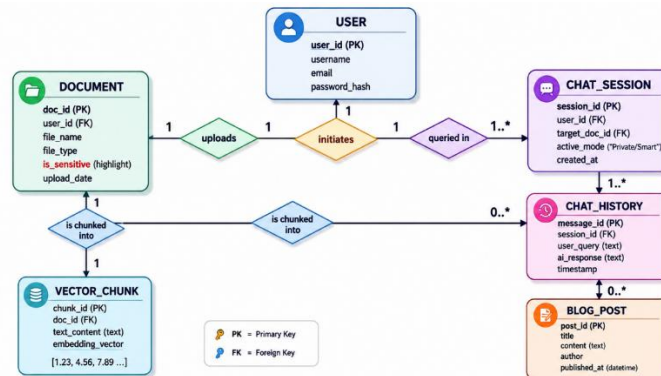


Figure: Entity-Relationship Diagram for Hybrid RAG Framework

Figures 5.8.2 ER Diagram

5.9 User Interface and Screenshots

The user interface serves as the interaction layer between users and the proposed Hybrid RAG system. It is designed to provide a simple, intuitive, and user-friendly environment for submitting queries and viewing generated responses. The interface allows users to enter questions through a text input field and receive context-aware answers generated by the system. The main interface consists of a query input section, response display area, and feedback component. When a user submits a query, the system processes the request and displays the

generated response along with relevant contextual information. The interface may also provide retrieval details, source references, or confidence indicators to improve transparency and user trust.

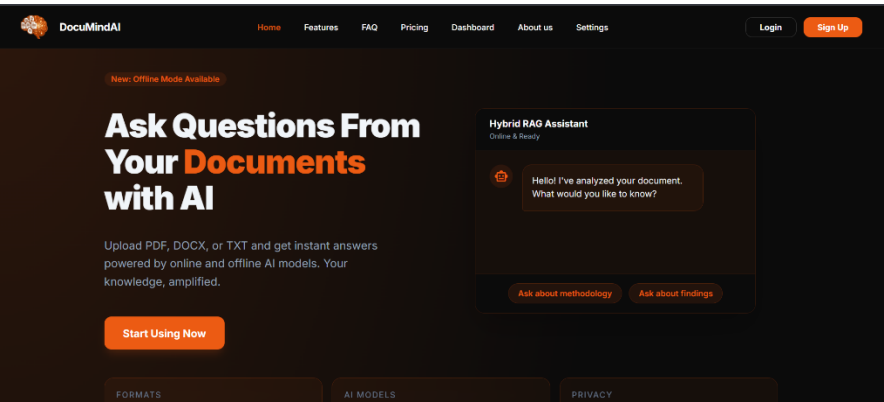


Figure 5.9.1: Illustrates the homepage of the Hybrid RAG Knowledge Assistant.

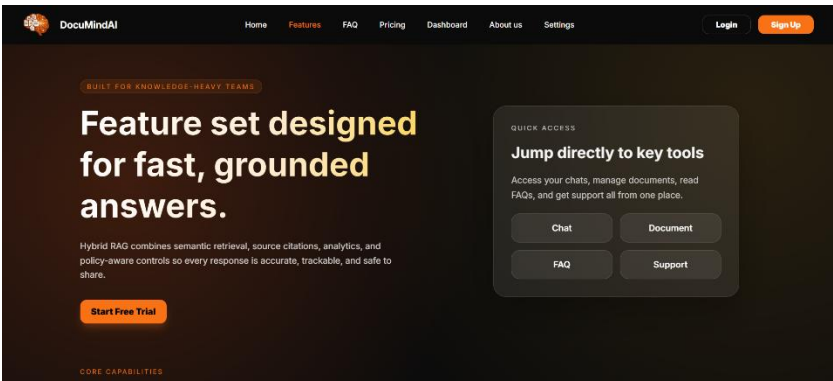


Figure 5.9.2: Illustrates the features of the Hybrid RAG Knowledge Assistant.

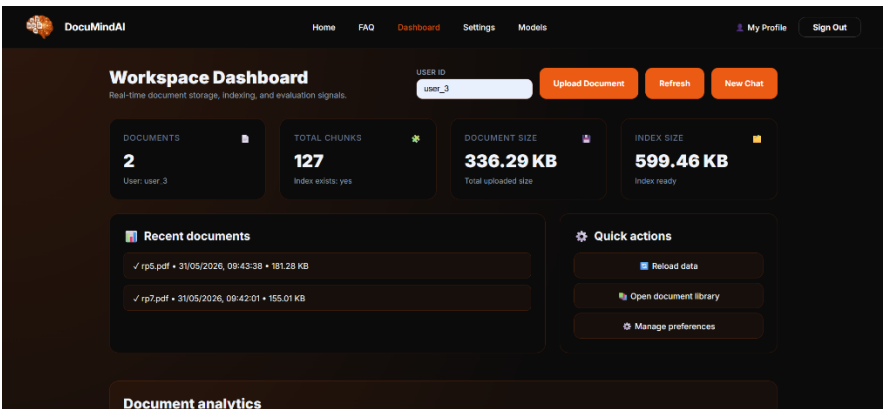


Figure 5.9.3: Illustrates the dashboard of the Hybrid RAG Knowledge Assistant.

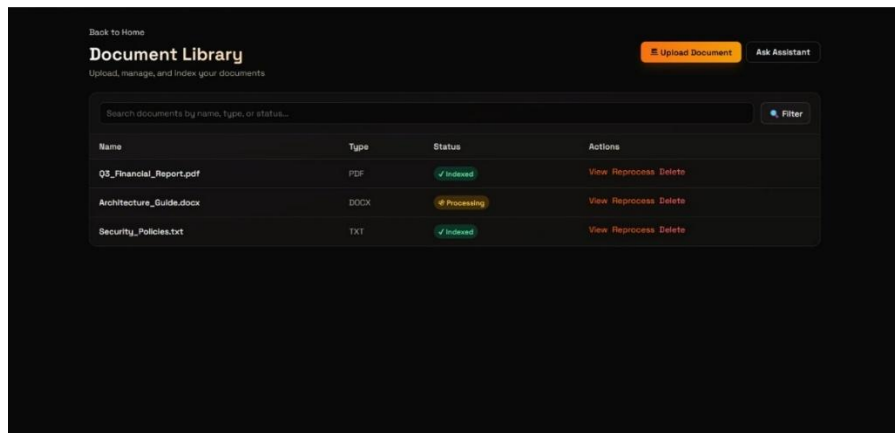


Figure 5.9.4: Illustrates the document library of the Hybrid RAG Knowledge Assistant.

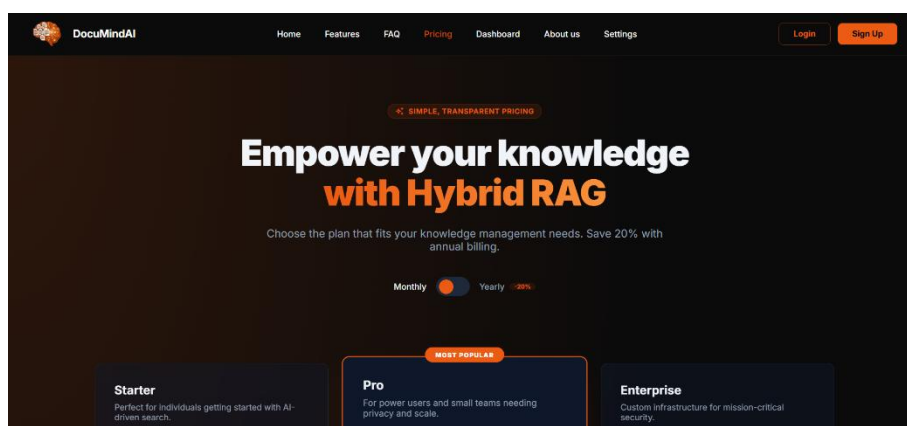


Figure 5.9.5: Illustrates the pricing of the Hybrid RAG Knowledge Assistant.

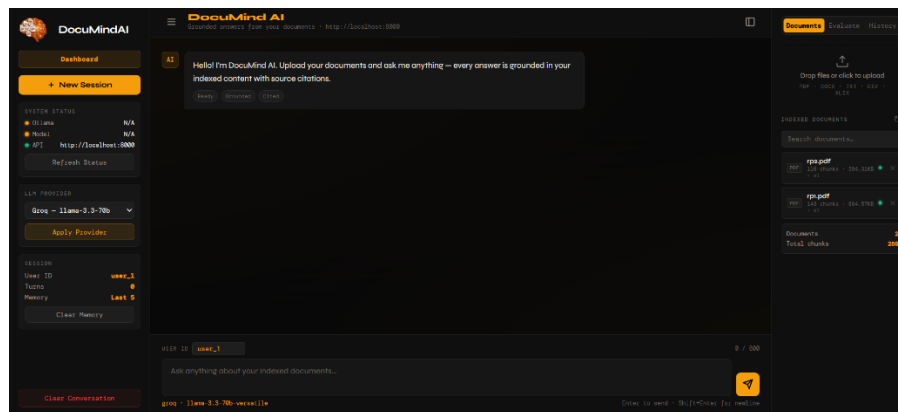


Figure 5.9.6: Illustrates the chat of the Hybrid RAG Knowledge Assistant.

CHAPTER 6

DEPLOYMENT MODELS

6.1 Introduction

Deployment plays a crucial role in determining the accessibility, performance, scalability, and security of an intelligent information retrieval system. The proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework can be deployed in different environments depending on organizational requirements, resource availability, and security constraints. This chapter discusses the deployment strategies adopted for the proposed system, including offline deployment and online cloud deployment. It also presents a comparative analysis of both approaches, examines security and privacy considerations, and evaluates the scalability of the framework. Understanding these deployment models is essential for selecting the most suitable implementation strategy for various real-world applications.

6.2 Comparative Analysis

Both offline and online cloud deployment models offer distinct advantages and limitations. The choice between these approaches depends on organizational requirements, security policies, budget constraints, and scalability needs.

Offline deployment provides superior data privacy and complete control over infrastructure. Since all data remains within the local environment, it is highly suitable for organizations handling sensitive information. It also eliminates dependency on internet connectivity and external service providers. However, offline deployment requires significant investment in hardware, maintenance, and technical expertise.

In contrast, online cloud deployment offers excellent scalability, flexibility, and accessibility. Organizations can quickly expand system resources and access advanced AI services without investing heavily in local infrastructure. Cloud deployment simplifies maintenance and supports global user access. However, it introduces concerns related to data privacy, recurring operational costs, and internet dependency.

6.3 Security and Privacy Considerations

Security and privacy are critical factors in the deployment of Retrieval-Augmented Generation systems. Since the proposed framework processes user queries and accesses external knowledge sources, protecting sensitive information is essential. Effective security mechanisms help prevent unauthorized access, data leakage, and cyber threats while ensuring compliance with regulatory requirements.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Experimental Setup

The experimental setup was designed to evaluate the performance and effectiveness of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework in intelligent question-answering tasks. The evaluation focused on measuring retrieval accuracy, response relevance, and overall system efficiency by comparing the proposed hybrid retrieval approach with individual retrieval techniques.

The experiments were conducted using a knowledge base consisting of research papers, technical documentation, educational resources, and domain-specific textual documents related to Artificial Intelligence, Natural Language Processing, and Retrieval-Augmented Generation. These documents were collected from reliable sources and stored in digital formats such as PDF and text files. Before retrieval, all documents underwent preprocessing, including text extraction, normalization, chunking, and metadata generation. The processed text segments were converted into vector embeddings and stored within a ChromaDB vector database.

The implementation environment consisted of Python as the primary programming language, along with LangChain, Sentence Transformers, ChromaDB, NumPy, and Pandas libraries. The system was developed and tested on a Windows-based platform equipped with an Intel Core i5 processor, 8 GB RAM, and SSD storage.

7.2 Evaluation Metrics

The performance of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework was evaluated using multiple quantitative metrics that measure both retrieval effectiveness and response quality. These metrics provide a comprehensive assessment of the system's ability to retrieve relevant information, generate accurate responses, and maintain efficient performance. The selected evaluation metrics include Precision, Recall, F1-Score, Response Relevance, and Latency. Together, these metrics help determine the overall effectiveness of the proposed Hybrid RAG framework in intelligent question-answering applications.

7.2.1 Precision

Precision measures the proportion of retrieved documents that are relevant to the user query. It evaluates the accuracy of the retrieval process by determining how many of the retrieved documents actually contain useful information. A high precision value indicates that the retrieval system minimizes irrelevant results and provides highly relevant information for response generation.

Precision is calculated using the following formula:

$$\text{Precision} = \text{Relevant Retrieved Documents} / \text{Total Retrieved Documents}$$

In the context of the proposed Hybrid RAG framework, high precision ensures that the Large Language Model receives accurate contextual information, thereby improving the quality and reliability of generated responses.

7.2.2 Recall

Recall measures the ability of the retrieval system to identify all relevant documents available within the knowledge base. Unlike precision, which focuses on accuracy, recall evaluates the completeness of the retrieval process. A higher recall value indicates that the system successfully retrieves a larger proportion of relevant information.

Recall is calculated as:

$$\text{Recall} = \text{Relevant Retrieved Documents} / \text{Total Relevant Documents}$$

High recall is important because missing relevant documents may result in incomplete context and less informative responses. Therefore, the proposed framework aims to achieve a balance between precision and recall.

7.2.3 F1-Score

The F1-Score is a combined metric that considers both precision and recall. It provides a balanced measure of retrieval performance by calculating the harmonic mean of precision and recall. This metric is particularly useful when both retrieval accuracy and retrieval completeness are equally important.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A higher F1-Score indicates that the system effectively balances precision and recall while maintaining overall retrieval quality.

7.2.4 Response Relevance

Response Relevance measures how closely the generated response aligns with the user's query and information requirements. This metric evaluates the contextual appropriateness, semantic correctness, and factual consistency of the generated answer. Response relevance is particularly important in Retrieval-Augmented Generation systems because the ultimate objective is not only to retrieve information but also to generate meaningful and useful responses.

Higher response relevance indicates that the system successfully understands user intent and provides accurate, context-aware answers. This metric may be assessed through manual evaluation, user feedback, or automated semantic similarity measures.

7.2.5 Latency

Latency measures the total time required by the system to process a query, retrieve relevant information, and generate a response. It reflects the efficiency and responsiveness of the framework. Lower latency values indicate faster system performance and a better user experience.

Latency is calculated as:

$$\text{Latency} = \text{Response Generation Time} - \text{Query Submission Time}$$

The proposed Hybrid RAG framework aims to minimize latency while maintaining high retrieval accuracy and response quality. Efficient retrieval mechanisms, optimized vector search operations, and adaptive retrieval strategies contribute to reducing overall response time. These evaluation metrics collectively provide a comprehensive framework for assessing the effectiveness, efficiency, and reliability of the proposed Adaptive Hybrid Retrieval-Augmented Generation system.

7.3 Experimental Results

The experimental evaluation was conducted to assess the effectiveness of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework in retrieving relevant information and generating accurate responses. The performance of the proposed framework was compared with standalone Sparse Retrieval (BM25) and Dense Retrieval (Vector Search) approaches. The evaluation focused on retrieval quality, response relevance, and system efficiency using the metrics discussed in the previous section.

A diverse set of test queries was used during experimentation, including factual questions, conceptual explanations, technical information requests, and multi-step reasoning tasks. Each query was processed using the three retrieval approaches, and the results were recorded for comparative analysis. The obtained results demonstrate that the Hybrid RAG framework consistently outperformed individual retrieval methods across most evaluation metrics.

Retrieval Performance Results

The retrieval performance was evaluated using Precision, Recall, and F1-Score. The results obtained from the experiments are presented in Table 7.1.

Table 7.1: Retrieval Performance Comparison

Method	Precision	Recall	F1-Score
BM25 (Sparse Retrieval)	0.81	0.75	0.78
Dense Retrieval	0.84	0.82	0.83
Proposed Hybrid RAG	0.91	0.88	0.89

The results indicate that the proposed Hybrid RAG framework achieved the highest precision, recall, and F1-score values. The integration of sparse and dense retrieval enabled the system to leverage both keyword matching and semantic understanding, resulting in improved retrieval effectiveness.

Response Relevance Evaluation

Response relevance was evaluated using a five-point scoring scale, where higher scores indicate better contextual alignment and factual accuracy. The average response relevance scores are shown in Table 7.2.

Table 7.2: Response Relevance Comparison

Method	Response Relevance Score
BM25 (Sparse Retrieval)	4.0 / 5
Dense Retrieval	4.3 / 5
Proposed Hybrid RAG	4.7 / 5

The Hybrid RAG framework achieved the highest relevance score due to its ability to combine lexical and semantic retrieval information before response generation. The context fusion mechanism further improved the quality of contextual information supplied to the Large Language Model.

Latency Analysis

System latency was measured as the average time required to process a query and generate a response. The results are presented in Table 7.3.

Table 7.3: Average Latency Comparison

Method	Average Latency (Seconds)
BM25 (Sparse Retrieval)	1.1
Dense Retrieval	1.8
Proposed Hybrid RAG	2.0

Although the proposed framework exhibited slightly higher latency compared to individual retrieval approaches, the increase was relatively small considering the significant improvements in retrieval quality and response relevance. The adaptive retrieval strategy helped minimize unnecessary processing while maintaining high performance.

Sample Query Evaluation

Several representative queries were tested to evaluate practical system performance. For factual queries, the Hybrid RAG framework achieved high retrieval precision and generated concise, accurate responses. For explanatory and technical queries, the system successfully combined information from multiple sources to provide comprehensive answers. In multi-step reasoning tasks, the adaptive retrieval mechanism selected relevant documents from both sparse and dense retrieval channels, resulting in improved contextual understanding and response quality.

Overall Performance

The experimental results demonstrate that the proposed Adaptive Hybrid Retrieval-Augmented Generation framework effectively combines the strengths of sparse retrieval, dense retrieval, and adaptive retrieval selection. Compared with standalone retrieval approaches, the framework achieved higher retrieval accuracy, better contextual relevance, and more reliable response generation. These findings validate the effectiveness of the proposed architecture and highlight its suitability for intelligent question-answering and knowledge retrieval applications across diverse domains.

7.4 Comparative Performance Analysis

The comparative performance analysis was conducted to evaluate the effectiveness of the proposed Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework against standalone Sparse Retrieval (BM25) and Dense Retrieval (Vector Search) approaches. The comparison focused on retrieval accuracy, response relevance, contextual understanding, and system efficiency. The objective was to determine whether combining multiple retrieval techniques provides measurable improvements over individual retrieval methods.

The experimental results demonstrated that BM25 performed effectively for keyword-based queries where exact term matching was essential. It achieved satisfactory precision and low response latency due to its computational efficiency. However, BM25 struggled with semantically complex queries because it relied primarily on lexical matching and could not effectively capture contextual relationships between terms.

7.5 Discussion of Findings

The findings obtained from the experimental evaluation indicate that the proposed Adaptive Hybrid Retrieval-Augmented Generation framework successfully addresses several limitations associated with traditional retrieval systems and standalone Large Language Models. The integration of sparse retrieval, dense retrieval, adaptive retrieval selection, and retrieval-grounded generation contributed significantly to improved retrieval effectiveness and response quality.

One of the most important observations is that hybrid retrieval consistently outperformed individual retrieval approaches. Sparse retrieval was highly effective for exact keyword matching, while dense retrieval provided superior semantic understanding. The combination of these complementary techniques enabled the framework to retrieve more comprehensive and relevant information across different query types.

7.6 Limitations of the System

Despite its improved performance, the proposed Adaptive Hybrid Retrieval-Augmented Generation framework has several limitations. First, the quality of generated responses depends heavily on the quality and coverage of the knowledge base. Incomplete or outdated documents may negatively affect retrieval accuracy and response relevance.

Second, the framework requires additional computational resources compared to standalone retrieval methods because it performs both sparse and dense retrieval operations. This may increase processing time and resource consumption for large-scale deployments.

Third, the effectiveness of dense retrieval depends on the quality of embedding models.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The rapid advancement of Artificial Intelligence and Large Language Models has transformed the way users interact with information systems. However, traditional language models face several challenges, including hallucination, limited access to real-time information, and difficulty handling domain-specific knowledge. These limitations highlight the need for intelligent systems capable of combining powerful language generation capabilities with reliable knowledge retrieval mechanisms. To address these challenges, this project proposed an **Adaptive Hybrid Retrieval-Augmented Generation (RAG) Framework** designed to improve retrieval accuracy, contextual understanding, and response reliability.

The proposed framework integrates sparse retrieval, dense retrieval, adaptive retrieval strategies, context fusion mechanisms, and Large Language Models within a unified architecture. Sparse retrieval based on BM25 enables effective keyword matching, while dense retrieval utilizes vector embeddings and semantic search to identify contextually relevant information. The adaptive retrieval strategy dynamically balances these retrieval methods according to query characteristics, ensuring efficient and accurate information retrieval. Retrieved documents are further processed through reranking and context fusion modules before being supplied to the Large Language Model for response generation.

A detailed study of Natural Language Processing, Information Retrieval Systems, Vector Databases, Embedding Models, and Large Language Models was conducted to establish the theoretical foundation of the framework. Existing research on Retrieval-Augmented Generation systems was reviewed, and several research gaps were identified. Based on these observations, the proposed Hybrid RAG architecture was developed to address shortcomings associated with conventional retrieval and generation systems.

8.2 Key Contributions

The major contributions of this project are summarized as follows:

1. **Development of a Hybrid Retrieval Framework:** The project proposed a novel Adaptive Hybrid Retrieval-Augmented Generation architecture that combines sparse retrieval and dense retrieval techniques within a unified system.
2. **Integration of Adaptive Retrieval Strategy:** An adaptive retrieval mechanism was designed to dynamically select and balance retrieval methods based on query characteristics and contextual requirements.
3. **Enhanced Context Construction and Fusion:** The framework incorporates context fusion and reranking mechanisms to improve the quality of information supplied to the Large Language Model.
4. **Reduction of Hallucination:** By grounding response generation in retrieved knowledge, the framework improves factual accuracy and reduces the occurrence of hallucinated responses.
5. **Comprehensive Evaluation:** The proposed system was evaluated using multiple performance metrics, including Precision, Recall, F1-Score, Response Relevance, and Latency, demonstrating its effectiveness compared to standalone retrieval approaches.

These contributions collectively advance the development of intelligent and context-aware Retrieval-Augmented Generation systems.

8.3 Future Scope

Although the proposed Adaptive Hybrid Retrieval-Augmented Generation framework achieved promising results, several opportunities exist for future enhancement and research. One potential improvement is the integration of more advanced embedding models and Large Language Models capable of providing better semantic understanding and reasoning

capabilities. As AI technology continues to evolve, future systems can leverage next-generation models to improve response quality and contextual awareness.

Another area of future research involves the incorporation of multimodal retrieval capabilities. In addition to textual information, future RAG systems may retrieve and process images, audio, video, and structured data sources. This would enable more comprehensive knowledge retrieval and support a wider range of applications.

8.4 Final Remarks

The proposed Adaptive Hybrid Retrieval-Augmented Generation Framework represents an effective approach for improving intelligent question-answering systems through the integration of retrieval mechanisms and Large Language Models. By combining lexical retrieval, semantic retrieval, adaptive retrieval selection, and retrieval-grounded response generation, the framework delivers accurate, context-aware, and reliable responses. The successful completion of this project demonstrates the potential of Hybrid RAG systems in addressing the limitations of traditional language models and highlights their importance in the future development of intelligent Artificial Intelligence applications.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992, 2019.
- [7] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” in *Proceedings of EMNLP*, pp. 2383–2392, 2016.
- [8] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, et al., “LLaMA: Open and Efficient Foundation Language Models,” arXiv preprint arXiv:2302.13971, 2023.
- [9] OpenAI, “GPT-4 Technical Report,” arXiv preprint arXiv:2303.08774, 2023.
- [10] J. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” in *Proceedings of ICML*, pp. 3929–3938, 2020.
- [11] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in *Proceedings of SIGIR*, pp. 39–48, 2020.

- [12] Y. Karpukhin, V. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proceedings of EMNLP*, pp. 6769–6781, 2020.
- [13] Z. Jiang, F. Xu, J. Gao, Z. Li, and H. Zhang, “Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity,” in *Proceedings of NAACL*, 2024.
- [14] X. Fan, Y. Ding, K. Wang, and X. Li, “A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models,” arXiv preprint arXiv:2405.06211, 2024.
- [15] M. Abo El-Enen, A. Mohamed, and S. Ibrahim, “A Survey on Retrieval-Augmented Generation Models for Healthcare Applications,” arXiv preprint, 2025.
- [16] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [17] LangChain Documentation, “Building Applications with Large Language Models,” 2025.
- [18] ChromaDB Documentation, “Open-Source Embedding Database,” 2025.
- [19] Hugging Face, “Transformers: State-of-the-Art Natural Language Processing,” 2025.
- [20] Google Research, “Gemini: A Family of Highly Capable Multimodal Models,” 2024.

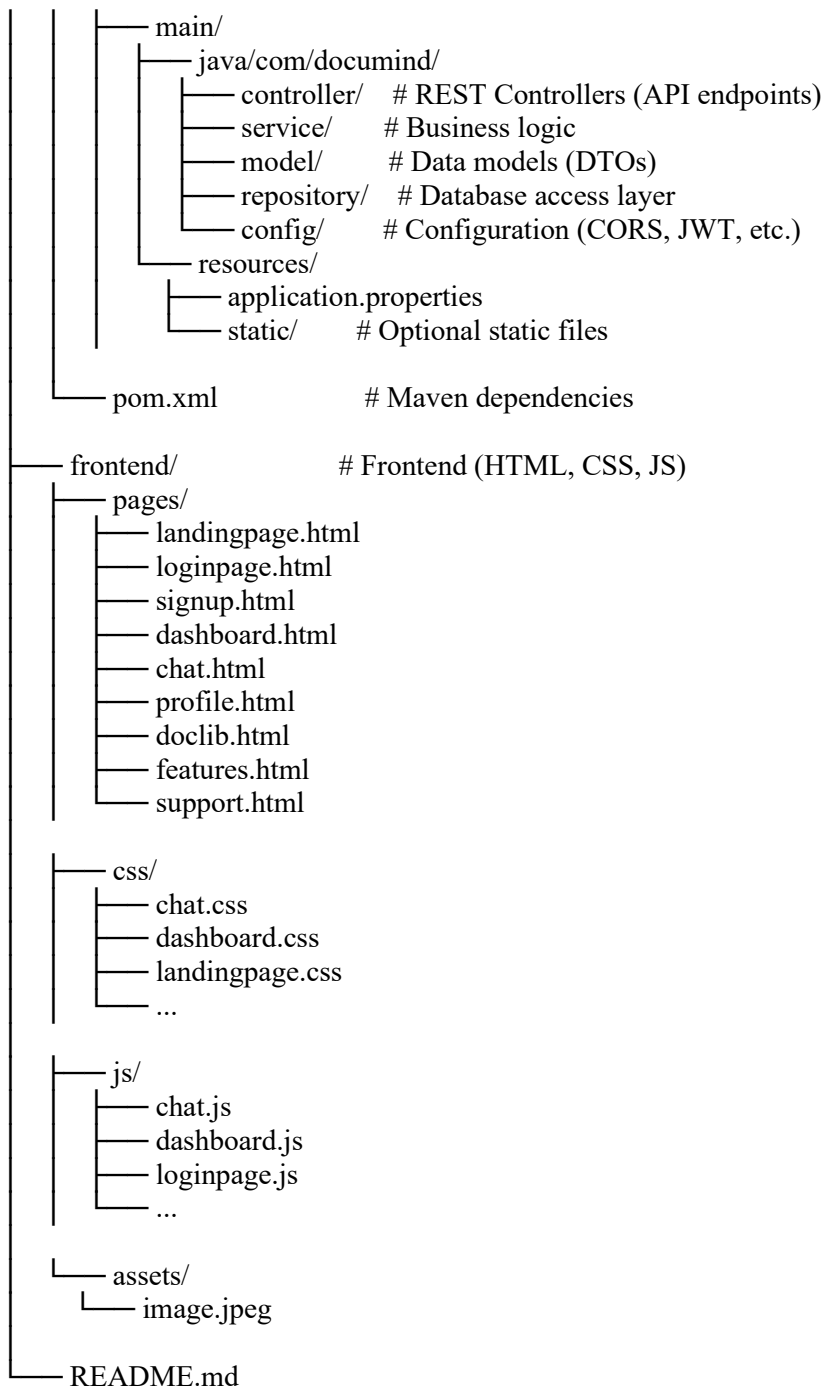
APPENDIX A

Research Highlights

- Proposed an Adaptive Hybrid RAG framework integrating sparse and dense retrieval.
- Developed a dynamic retrieval strategy based on query complexity analysis.
- Improved retrieval accuracy through context fusion and reranking mechanisms.
- Reduced hallucination in LLM responses using retrieval-grounded generation.
- Enabled scalable deployment in both offline and cloud-based environments.

File Structure

```
Hybrid-rag-knowledge-assistant/
├── ai_service/                # Python FastAPI + LangChain AI Service
│   ├── main.py               # FastAPI entry point
│   ├── rag_pipeline.py       # Core RAG pipeline logic
│   ├── retriever.py          # Hybrid retrieval (FAISS + BM25)
│   ├── query_processor.py     # Query preprocessing
│   ├── document_processor.py  # PDF processing & chunking
│   ├── embeddings.py          # Embedding model handler
│   ├── llm_manager.py         # LLM provider management
│   ├── ollama_manager.py      # Local LLM (Ollama) handler
│   ├── guardrails.py          # Safety & validation checks
│   ├── memory_manager.py      # Conversation memory
│   ├── prompts.py             # Prompt templates
│   ├── config.py              # Configuration settings
│   ├── config_router.py       # Provider routing logic
│   ├── model_registry.py      # Model configuration registry
│   ├── user_config_manager.py  # User-specific settings
│   └── requirements.txt       # Python dependencies
├── user_configs/              # Per-user configurations
│   └── user_1.json
├── uploads/                   # Uploaded documents
│   └── vector_store/          # FAISS vector database
│       ├── user_user_1/
│       │   └── index.faiss
│       ├── user_user_2/
│       │   └── index.faiss
├── model_cache/               # Cached ML models
├── springboot_backend/        # Java Spring Boot Backend
└── src/
```



APPENDIX B

Source Code

```
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from config_router import router as config_router
import shutil
from llm_manager import clear_llm_cache, get_current_provider_info
import os
from rag_pipeline import ingest_document, ask_question, reset_user_session

app = FastAPI(title="DocuMind AI Service", version="3.0")

app.add_middleware(
    CORSMiddleware, allow_origins=["*"],
    allow_credentials=True, allow_methods=["*"], allow_headers=["*"]
)
# ☒ Mount config router — adds all /config/* endpoints
app.include_router(config_router)

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.get("/")
def read_root():
    """Health check endpoint."""
    return {"status": "DocuMind AI Service is running.", "version": "2.0"}

@app.post("/upload")
async def upload_file(
    user_id: str = Form(...),
    file: UploadFile = File(...)
):
    """
    Upload a PDF and process it into the FAISS vector database.
    """
    if not file.filename.endswith('.pdf'):
        raise HTTPException(status_code=400, detail="Only PDF files are supported.")

    file_path = os.path.join(UPLOAD_DIR, f'{user_id}_{file.filename}')

    try:
        # Save file temporarily
        with open(file_path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)

        # ☒ Use ingest_document from rag_pipeline (returns a dict now)
        result = ingest_document(file_path, user_id)

        if result["status"] == "error":
            raise HTTPException(status_code=500, detail=result["message"])
```

```

        return {
            "status": "success",
            "message": f'Processed '{file.filename}' successfully.',
            "chunks_created": result["chunks_created"],
            "user_id": user_id
        }

    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
    finally:
        # Clean up temp file after processing
        if os.path.exists(file_path):
            os.remove(file_path)

@app.post("/ask")
async def ask(
    query: str = Form(...),
    user_id: str = Form(...)
):
    result = ask_question(query, user_id)

    # ☒ Guardrail blocked — return 200 with explanation, not 500
    if result["status"] == "blocked":
        return {
            "status": "blocked",
            "reason": result.get("reason"),
            "answer": result["answer"],
            "sources": []
        }

    if result["status"] == "error":
        raise HTTPException(status_code=500, detail=result["answer"])

    return {
        "status": "success",
        "answer": result["answer"],
        "sources": result.get("sources", []),
        "original_query": result.get("original_query", query),
        "processed_query": result.get("processed_query", query)
    }

@app.get("/provider")
def get_provider():
    """Returns the currently active LLM provider."""
    return get_current_provider_info()

@app.post("/provider/switch")
async def switch_provider(
    provider: str = Form(...), # "offline" | "online"
    online_provider: str = Form(None) # "groq" | "openai" | "anthropic"
):
    """
    Switches LLM provider at runtime without restarting the server.
    """
    valid_providers = ["offline", "online"]
    valid_online = ["groq", "openai", "anthropic"]

```

```

if provider not in valid_providers:
    raise HTTPException(
        status_code=400,
        detail=f"Invalid provider. Choose from: {valid_providers}"
    )
if provider == "online" and online_provider not in valid_online:
    raise HTTPException(
        status_code=400,
        detail=f"Invalid online provider. Choose from: {valid_online}"
    )

# ☒ Update environment variables at runtime
os.environ["LLM_PROVIDER"] = provider
os.environ["ONLINE_PROVIDER"] = online_provider or "groq"

# ☒ Reload config values
import config
config.LLM_PROVIDER = provider
config.ONLINE_PROVIDER = online_provider or "groq"

# ☒ Clear cache so next request uses new provider
clear_llm_cache()

return {
    "status": "success",
    "message": f"Switched to {provider} ({online_provider or 'ollama'})",
    "provider": provider,
    "online_provider": online_provider
}
@app.post("/reset")
async def reset_session(
    user_id: str = Form(...)
):
    try:
        result = reset_user_session(user_id)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")
def health_check():
    from ollama_manager import is_ollama_running, is_model_available
    from config import OLLAMA_MODEL, LLM_PROVIDER

    provider_info = get_current_provider_info()
    ollama_ok = is_ollama_running() if LLM_PROVIDER == "offline" else None
    model_ok = is_model_available(OLLAMA_MODEL) if ollama_ok else None

    return {
        "status": "ok",
        "active_provider": provider_info,
        "ollama_running": ollama_ok,
        "ollama_model_ready": model_ok,
    }

```

ABSTRACT IN HINDI

बड़े भाषा मॉडल (Large Language Models - LLMs) के विकास ने प्राकृतिक भाषा प्रसंस्करण के क्षेत्र में महत्वपूर्ण प्रगति की है। ये मॉडल मानव-समान पाठ को समझने और उत्पन्न करने में सक्षम हैं, किंतु इनमें कई सीमाएँ भी विद्यमान हैं, जैसे गलत अथवा काल्पनिक जानकारी (Hallucination) उत्पन्न करना, अद्यतन ज्ञान की कमी, तथा डोमेन-विशिष्ट और वास्तविक समय की जानकारी तक सीमित पहुँच। इन चुनौतियों के समाधान हेतु Retrieval-Augmented Generation (RAG) एक प्रभावी तकनीक के रूप में उभरी है, जो बाहरी ज्ञान स्रोतों से जानकारी प्राप्त करके बड़े भाषा मॉडलों की उत्तर निर्माण क्षमता को सशक्त बनाती है।

यह परियोजना “बुद्धिमान प्रश्नोत्तर प्रणाली हेतु अनुकूली हाइब्रिड Retrieval-Augmented Generation (RAG) फ्रेमवर्क” प्रस्तुत करती है, जिसका उद्देश्य AI-आधारित उत्तरों की सटीकता, प्रासंगिकता तथा विश्वसनीयता को बढ़ाना है। प्रस्तावित प्रणाली में स्पार्स रिट्रीवल तकनीकों, जैसे BM25, तथा वेक्टर एम्बेडिंग आधारित डेंस रिट्रीवल विधियों को संयोजित किया गया है। यह संयोजन कीवर्ड-आधारित और अर्थ-आधारित (Semantic) खोज दोनों के लाभ प्रदान करता है, जिससे अधिक प्रासंगिक जानकारी प्राप्त की जा सकती है।

प्रस्तावित फ्रेमवर्क का मुख्य घटक एक अनुकूली रिट्रीवल तंत्र (Adaptive Retrieval Mechanism) है, जो उपयोगकर्ता के प्रश्न की जटिलता, संदर्भ और सूचना आवश्यकताओं का विश्लेषण कर सबसे उपयुक्त रिट्रीवल रणनीति का चयन करता है। स्थानीय दस्तावेजों, डेटाबेस तथा ऑनलाइन ज्ञान स्रोतों से प्राप्त जानकारी को एक Context Fusion Module के माध्यम से संसाधित किया जाता है। यह मॉड्यूल प्रासंगिक दस्तावेजों को रैंक, फ़िल्टर और संयोजित करके बड़े भाषा मॉडल के लिए उच्च-गुणवत्ता वाला संदर्भ तैयार करता है, जिससे उत्तरों की गुणवत्ता में सुधार होता है।

इसके अतिरिक्त, प्रणाली में Feedback एवं Continuous Learning Mechanism भी सम्मिलित है, जो उपयोगकर्ता प्रतिक्रिया और रिट्रीवल प्रदर्शन के आधार पर समय के साथ प्रणाली की दक्षता को बढ़ाता है। यह फ्रेमवर्क ऑफ़लाइन तथा ऑनलाइन क्लाउड दोनों प्रकार की तैनाती का समर्थन करता है, जिससे गोपनीयता-संवेदनशील वातावरण और वास्तविक समय की सूचना आवश्यकताओं दोनों को पूरा किया जा सकता है।

प्रस्तावित हाइब्रिड RAG फ्रेमवर्क का उद्देश्य Hallucination को कम करना, संदर्भगत समझ को बेहतर बनाना तथा अधिक सटीक और विश्वसनीय उत्तर उत्पन्न करना है। परिणाम दर्शाते हैं कि यह प्रणाली पारंपरिक रिट्रीवल और जनरेशन विधियों की तुलना में बेहतर उत्तर गुणवत्ता, उच्च अर्थगत प्रासंगिकता तथा बेहतर समग्र प्रदर्शन प्रदान करती है। इसलिए यह फ्रेमवर्क अगली पीढ़ी की AI-संचालित प्रश्नोत्तर, ज्ञान प्रबंधन और संवादात्मक प्रणालियों के लिए एक प्रभावी एवं स्केलेबल समाधान प्रस्तुत करता है।

APPENDIX – C

VITAE



The author Abhay Maurya was born on 31/03/2004 at Rudrapur. He passed his high school during the year 2020 and his Intermediate in 2022 from “Himalayan Progressive School, Kichha”

He joined as an under graduate student in Dr. APJ. Abdul Kalam Institute of Technology, Tanakpur, Champawat for program Bachelor of Technology (Computer Science & Engineering).

Address For Communication

Abhay Maurya

S/O Mr. Umesh Kumar Maurya

Kichha Pin code -263148

+91 8191826630

abhay.maurya0303@gmail.com

VITAE



The author, Amit Yadav was born on 29/03/2004 at Rudrapur. He passed his high school during the year 2020 and his Intermediate in 2022 from “Jaycees public School”.

He joined as an under graduate student in Dr. APJ. Abdul Kalam Institute of Technology, Tanakpur, Champawat for program Bachelor of Technology (Computer Science & Engineering).

Address For Communication

Amit Yadav

S/O Mr. Ram Pravesh Yadav

Kichha Pin code -263153

+917439327622

amity3289@gmail.com

VITAE



The author, Shanu Ahmed was born on 5/12/2005 at Almora. He passed his Diploma during the year 2023 from “Government Polytechnic Kanda Bageshwar”.

He joined as an under graduate student in Dr. APJ. Abdul Kalam Institute of Technology, Tanakpur, Champawat for program Bachelor of Technology (Computer Science & Engineering).

Address For Communication

Shanu Ahmed

S/O Mr. Shamshad Ahmed

Almora Pin code - 263601

+91 7055417849

shanuahmed03@gmail.com

VITAE



The author, Varun Rana was born on 22/04/2004 at Sitarganj. He passed his high school during the year 2019 and his Intermediate in 2021 from “P.P.S.V.M. Intercollege, Nanakmatta”.

He joined as an under graduate student in **Dr. APJ. Abdul Kalam Institute of Technology**, Tanakpur, Champawat for program Bachelor of Technology (Computer Science & Engineering).

Address For Communication

Varun Rana

S/O Mr. Ashok Singh Rana

Sitarganj, Pin code - 262405

+91 7500961740

varunrana1902@gmail.com

VITAE



The author, Sameer Ahmad was born on 10/07/2003 at Khatima. He passed his Diploma during the year 2023 from “Government Polytechnic, SakhtiFarm”.

He joined as an under graduate student in Dr. APJ. Abdul Kalam Institute of Technology, Tanakpur, Champawat for program Bachelor of Technology (Computer Science & Engineering).

Address For Communication

Samir Ahmad

S/O Mr. Naseem Ahmed

Khatima Pin code -262308

+91 8077705884

azharahmad5310@gmail.com